

Route Graduation Project
Intelligent RAG System

Level 1 Report: **Foundation RAG Pipeline**

Books to Scrape Dataset | 1,000 Records | Gemini API

Executive Summary

Level 1 implements a complete Retrieval-Augmented Generation (RAG) pipeline that answers user questions based on a dataset of 1,000 book records scraped from Books to Scrape. The pipeline consists of four sequential stages:

- Data Ingestion – load, clean, and structure the raw dataset
- Embedding & Indexing – generate semantic vectors and store them in a FAISS index
- Basic Retrieval – embed the user query and retrieve the top-5 most relevant chunks
- Answer Generation – pass retrieved context to Google Gemini with a grounded prompt

The system loads and cleans the data, chunks each book as a complete record, and generates semantic embeddings using the all-MiniLM-L6-v2 Sentence Transformer model. Vectors are stored in a FAISS IndexFlatL2 index for fast nearest-neighbour search. At query time the top-5 most relevant chunks, along with their similarity scores and metadata, are retrieved and forwarded to the Gemini API. A carefully designed prompt restricts the model to the retrieved context, producing grounded, hallucination-free answers.

The entire process is transparent: retrieved documents, similarity scores, metadata, and source URLs are displayed alongside the final answer. This report documents the design decisions, implementation details, and justifications for each component.

1. Data Ingestion

1.1 Loading the Dataset

The dataset was collected during Level 0 from the Books to Scrape website using a Python web scraper and saved as `books_dataset.json`. Each record contains the fields described in Table 1.1.

Field	Description
Title	Book title
Price	Price in GBP (e.g., £51.77)
Rating	Star rating expressed as a word (One – Five)
Description	Product description (plain text)
Source URL	URL of the book's individual page on Books to Scrape

Table 1.1 – Dataset Field Descriptions

The dataset was loaded using Python's `json` library and converted into a Pandas `DataFrame` for easier manipulation and cleaning:

```
import json
import pandas as pd
with open("books_dataset.json", "r", encoding="utf-8") as file:
    books = json.load(file)
df = pd.DataFrame(books)
```

Pandas simplifies text preprocessing, inspection, and chunk generation across all 1,000 records.

1.2 Data Cleaning and Preprocessing

To improve embedding quality, all text fields were cleaned using a custom `clean_text()` function. The following operations were applied sequentially:

- Remove newline characters (`\n`) – replaced with spaces to prevent broken text.
- Remove carriage returns (`\r`) – ensures consistent formatting across platforms.
- Collapse extra whitespace – multiple consecutive spaces reduced to a single space.
- Strip leading and trailing spaces – via `.strip()`.
- Convert all values to strings – prevents datatype inconsistencies during encoding.

Cleaning was applied to the title, description, price, and rating columns. Without this step, descriptions containing random line breaks or excess whitespace produce less reliable embedding vectors, which directly reduces retrieval accuracy.

1.3 Chunking Strategy

Selected Strategy: One Book = One Chunk

Each book record was treated as a single, self-contained chunk composed of all four fields:

```
Title: Sharp Objects Price: £47.82 Rating: Five Description: WICKED above her hipbone, GIRL across her heart...
```

Justification

After inspecting the dataset, it is evident that each book represents a complete and independent unit of information. Users are likely to query the book as a whole (e.g., “What is the price of Sharp Objects?” or “Which books have a five-star rating?”). Splitting the description into smaller pieces would separate the title, price, and rating from the descriptive text, weakening the semantic signal available to the retrieval model.

Alternatives Considered and Rejected

Chunking Method	Reason for Rejection
Sentence-based	Separates title/price/rating from description, losing essential context.
Paragraph-based	Most descriptions consist of a single paragraph; no benefit is gained.
Fixed-size (character count)	May truncate sentences mid-word, destroying semantic meaning.
Overlapping	Records are independent; overlap introduces redundancy without benefit.

Table 1.2 – Chunking Alternatives and Rejection Rationale

Weaknesses and Mitigations

Weakness	Mitigation
Larger chunks may include irrelevant content for narrow queries.	Descriptions are typically 150–300 words; chunk size remains manageable.
Similar books may produce similar embeddings, reducing diversity.	Semantic search ranks the most relevant chunk highest; metadata filtering can be added in later levels.
Individual sentences cannot be retrieved independently.	Acceptable trade-off because most queries target the book as a whole.

Table 1.3 – Chunking Weaknesses and Mitigations

1.4 Metadata Storage

For each chunk, the following metadata fields are stored alongside the chunk text to support transparent retrieval and downstream filtering:

Field	Purpose
record_id	Unique identifier (row index) for precise tracking.
title	Book title – displayed to the user in retrieval output.
price	Price – used to answer price-related queries.
rating	Rating – used to answer rating-related queries.
source_url	URL – enables source verification and transparency.

Table 1.4 – Stored Metadata Fields

2. Embedding & Indexing

2.1 Embedding Model Selection

The project uses the following Sentence Transformers model:

`sentence-transformers/all-MiniLM-L6-v2`

This model converts text into dense 384-dimensional vectors that capture semantic meaning. Table 2.1 details the justification for this selection.

Criterion	Evaluation
Language Support	Optimised for English, matching the dataset language.
Accuracy	Strong semantic retrieval – captures meaning rather than exact keywords (e.g., “cheap books” retrieves low-price records).
Speed	Lightweight; generates embeddings quickly even on modest hardware.
Cost	Free and open-source; no API costs – ideal for academic projects.
Embedding Dimension	384 dimensions provide a good trade-off between accuracy and storage/search efficiency.
Dataset Suitability	Performs well on short-to-medium texts – well-matched to book titles and descriptions.

Table 2.1 – Embedding Model Justification

2.2 Embedding Generation Process

Embeddings are generated in three steps:

1. Extract text from each chunk:

```
texts = [doc["text"] for doc in documents]
```

2. Generate embeddings using the model:

```
embeddings = embedding_model.encode(texts, show_progress_bar=True)
```

3. Convert to a NumPy float32 array for FAISS compatibility:

```
embeddings = np.array(embeddings).astype("float32")
```

The resulting array has shape (1000, 384), representing 1,000 book records each encoded as a 384-dimensional semantic vector.

2.3 Vector Store Selection: FAISS

FAISS (Facebook AI Similarity Search) was selected as the vector database. The selection is justified by the following characteristics:

- High-speed nearest-neighbour search – efficient even with thousands of vectors.
- Seamless Python integration – compatible with NumPy and the broader Python ecosystem.
- Memory efficiency – stores dense vectors compactly.
- Scalability – capable of handling significantly larger datasets.
- Wide adoption – well-documented and a standard component in RAG systems.

An IndexFlatL2 (Euclidean distance) index was created and populated:

```
dimension = embeddings.shape[1] # 384 index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
```

2.4 Saving the Index

To avoid recomputing embeddings on every run, the FAISS index is persisted to disk:

```
faiss.write_index(index, "books_faiss.index")
```

The index can subsequently be reloaded using `faiss.read_index()`, enabling fast startup for retrieval without re-encoding the corpus.

3. Basic Retrieval

3.1 Retrieval Pipeline

The retrieval stage follows these sequential steps:

- 4. The user enters a natural language question (e.g., “What is the price of A Light in the Attic?”).
- 5. The query is embedded using the same all-MiniLM-L6-v2 model used during indexing.
- 6. FAISS searches for the nearest vectors to the query embedding using L2 distance.
- 7. The top-k most similar chunks are retrieved.
- 8. Retrieved chunks, similarity scores, and metadata are displayed for transparency.

3.2 Query Embedding and Similarity Search

The query is encoded and searched against the FAISS index as follows:

```
query_embedding = embedding_model.encode([query]) distances, indices = index.search(query_embedding, k=5)
```

Distances are L2 (Euclidean) values; smaller values indicate higher semantic similarity between the query and the retrieved chunk.

3.3 Retrieval Parameter: Top-k = 5

The number of retrieved chunks (k) was set to 5 after evaluating the trade-offs shown in Table 3.1.

k Value	Pros	Cons
3	Faster generation; smaller prompt size.	May omit relevant context for broader queries.
5 (selected)	Balances retrieval quality, context size, speed, and prompt length.	No significant drawbacks.
10	Provides broader context coverage.	Introduces irrelevant content; increases token usage and response latency.

Table 3.1 – Retrieval Parameter Trade-offs

3.4 Displaying Retrieved Documents

For each retrieved chunk, the system outputs the following information to ensure transparency and allow users to verify answer grounding:

- Chunk text (title, price, rating, description)
- Similarity score (L2 distance)
- Metadata (record ID, title, price, rating, source URL)

4. Answer Generation Using Gemini API

4.1 Model Selection: Google Gemini

Google Gemini was selected as the Large Language Model for final answer generation. The selection is justified by the following capabilities:

- High-quality natural language generation with strong instruction-following.
- Fast response times suitable for interactive query answering.
- Simple integration via the google-generativeai Python library.
- Free access available for educational and academic projects.

4.2 Context Construction

All retrieved chunks are concatenated into a single context string, which is passed to Gemini alongside the user question:

```
context = "" for doc in retrieved_docs:     context += doc["chunk"] + "\n\n"
```

4.3 Prompt Design

A carefully structured prompt instructs Gemini to answer exclusively from the retrieved context and to avoid fabricating information. The prompt template is as follows:

```
You are a Retrieval-Augmented Generation (RAG) assistant. Use ONLY the information
provided in the context below. Rules: 1. Answer only from the retrieved context.
2. Do not use outside knowledge. 3. If the answer is not available in the context,
reply exactly: "I couldn't find the answer in the retrieved documents." 4.
Mention only facts supported by the retrieved chunks. Retrieved Context: {context}
User Question: {query} Answer:
```

4.4 Grounding Strategy and Hallucination Reduction

The following design decisions combine to minimise hallucination:

- The context is restricted exclusively to the top-5 retrieved chunks.
- Explicit prompt instructions require Gemini to ground all claims in that context.
- A fixed fallback response is specified for unanswerable queries, preventing fabricated answers.
- Source titles and URLs are displayed after the answer, enabling user cross-verification.

4.5 Final Output Structure

For every query, the system produces the following output:

- The user question
- Each retrieved chunk with its similarity score and metadata
- The final answer generated by Gemini
- The source titles and URLs used

This output structure makes the entire RAG process transparent and auditable.

5. Sample Query and Output

Query: “What is the price of A Light in the Attic?”

5.1 Retrieved Documents

Rank	Title	Price	Rating	Score (L2)
1	A Light in the Attic	£51.77	Three	1.025
2	The Light of the Fireflies	£54.43	One	1.128
3	Alice in Wonderland	£55.53	One	1.145
4	In a Dark, Dark Wood	£19.63	One	1.181
5	Amatus	£50.54	Five	1.231

Table 5.1 – Retrieved Chunks for Sample Query

5.2 Generated Answer

| *The price of A Light in the Attic is £51.77.*

5.3 Sources

- A Light in the Attic – http://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html

The top-ranked result corresponds exactly to the queried book, confirming correct semantic retrieval. The L2 distance of 1.025 is meaningfully lower than the next result (1.128), demonstrating that the embedding model correctly identifies the most semantically relevant chunk.

Conclusion

Level 1 successfully delivers a complete, transparent, and efficient Retrieval-Augmented Generation pipeline built on a 1,000-record Books to Scrape dataset. The key achievements of this phase are summarised below:

Component	Achievement
Data Ingestion	1,000 book records loaded, cleaned, and chunked as semantically complete units.
Embedding & Indexing	all-MiniLM-L6-v2 generates 384-dimensional vectors; FAISS IndexFlatL2 enables fast retrieval.
Basic Retrieval	Top-5 chunks retrieved with L2 similarity scores, chunk text, and full metadata.
Answer Generation	Google Gemini produces grounded, hallucination-free answers via a carefully designed prompt.

Table C.1 – Level 1 Achievement Summary

The pipeline is fully transparent: retrieved documents, similarity scores, metadata, and source URLs are displayed alongside every generated answer, enabling users to verify the grounding of each response.

This foundation prepares the system for the advanced features planned in subsequent levels. Level 2 will introduce query intelligence (intent detection, query expansion, and query rewriting), while Level 3 will add hybrid search combining dense and sparse retrieval methods. The modular architecture established in Level 1 is designed to accommodate these enhancements without requiring structural changes to the core pipeline.